

目 录

xv6-2020 实验环境搭建	1
实验一: Xv6 and Unix Utilities	2
实验二: System Calls.....	5
实验三: Page Tables	8
实验四: Traps.....	11
实验五: Lazy Page Allocation	14
实验六: Copy-on-Write Fork.....	18
实验七: Multithreading.....	21
实验八: Locks.....	25
实验九: File System.....	28
实验十: mmap	31
实验十一: Networking	34
总结与收获	37

xv6-2020 实验环境搭建

1. 搭建过程

本次 xv6-2020 实验需要在 Ubuntu 虚拟机中完成。由于 xv6-2020 面向 RISC-V 架构，因此首先安装 Linux 下的基础开发工具，包括 Git、Make、GCC 等，并配置 RISC-V 交叉编译工具链，用于将 xv6 源代码编译为能够在 RISC-V 平台运行的程序。

随后获取 MIT 官方提供的 xv6-riscv 2020 实验源码，并配置 QEMU 作为 RISC-V 硬件模拟器。由于系统自带或较新版本的 QEMU 与 xv6-2020 实验环境存在一定兼容性问题，最终选择源码编译 QEMU 5.1.0。

完成编译后，将 QEMU 安装到用户目录，并确保 xv6 使用该版本的 qemu-system-riscv64。之后进入 xv6 源码目录，通过 make 指令完成内核、用户程序和文件系统镜像的编译，再执行 make qemu 启动 xv6。最终能够正常看到 xv6 内核启动信息并进入 shell，说明 RISC-V 编译工具链、QEMU 和 xv6 源码环境均配置成功。

在完成基本运行环境后，我使用 Git 对整个实验进行版本管理。主分支主要保存实验报告和评分结果，每个实验分别建立独立分支，这样可以使不同实验之间的代码相互隔离，避免后续实验对前面已经完成的实验产生影响，同时方便保存和管理各实验的最终代码。

2. 遇到的问题及解决

问题一：QEMU 版本与 xv6-2020 存在兼容性问题

最初使用现有 QEMU 环境运行 xv6 时，出现系统启动异常，无法稳定完成内核初始化并进入 xv6 shell。

经过分析，xv6 属于直接与模拟硬件进行交互的操作系统，对 QEMU 的 RISC-V 设备模型和版本具有一定依赖。为了尽可能与 xv6-2020 发布时期的实验环境保持一致，最终重新下载并源码编译 QEMU 5.1.0。

重新编译并配置完成后，xv6 可以正常启动并进入 shell，问题得到解决。

问题二：向 GitHub 推送代码时出现网络和 TLS 错误

实验代码在本地能够正常提交，但执行 git push 时曾出现 TLS/GnuTLS 握手失败等网络错误。

经过分析，由于 Ubuntu 运行在 VMware 虚拟机中，而网络代理运行在 Windows 宿主机的 Clash Verge Rev 中，因此宿主机能够访问 GitHub 并不代表虚拟机中的 Git 可以自动使用相同代理。

解决过程中，我依次检查了：Ubuntu 虚拟机网络连接；Git 的 HTTP/HTTPS 代理配置；VMware 网络模式；Clash 代理状态。通过重新检查和配置网络以及 Git remote，最终能够正常将各实验分支推送至 GitHub。

实验一：Xv6 and Unix Utilities

1. 实验目的

本实验是 xv6-2020 系列实验的第一个正式实验，主要目的是熟悉 xv6 用户态程序的编写、编译和运行流程，并通过实现若干常见 Unix 工具，加深对进程、管道、文件系统和系统调用的理解。

本实验主要完成 sleep、pingpong、primes、find、xargs 等用户程序。通过这些程序，可以逐步掌握 xv6 中 fork、exec、wait、pipe、read、write、open 等系统调用的基本使用方法，并理解 Unix 系统中“通过小程序和管道组合完成复杂功能”的设计思想。

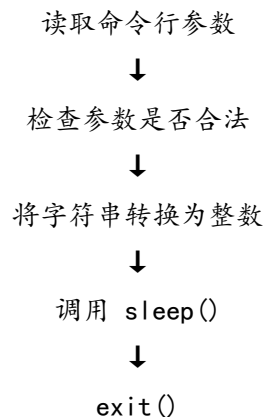
2. 实验内容

2.1 sleep

sleep 程序用于暂停指定数量的时钟节拍。

实现过程中，首先读取命令行参数，并判断参数数量是否正确；随后使用 atoi()将字符串参数转换为整数，调用 xv6 已提供的 sleep()系统调用暂停当前进程，最后正常退出。

程序的基本执行过程为：



2.2 pingpong

pingpong 要求父子进程之间通过管道传递数据，实现一次“ping-pong”通信。

首先创建两个管道，分别用于：父进程向子进程发送数据；子进程向父进程返回数据。随后调用 fork()创建子进程。父进程向管道中写入一个字节，子进程通过 read()接收数据后输出 received ping，再通过另一个管道向父进程发送响应。父进程收到数据后输出 received pong。

2.3 primes

primes 是本实验中较有代表性的程序，要求使用多个进程和管道实现埃拉托斯特尼筛法。

主进程首先将 2~35 的整数写入管道。之后，每一级筛选进程从输入管道中读取第一个整数，将其作为当前质数并输出，然后创建下一条管道，将不能被该质数整除的数字传递给下一个进程。

实现时主要步骤包括：主进程建立第一个管道；向管道中写入 2~35；子进程读取第一

个数字并将其作为质数；创建新的管道；将不能被当前质数整除的数据写入下一条管道；递归或循环创建下一级筛选进程；当管道中没有数据时结束。

2.4 find

`find` 程序用于递归查找指定目录下具有特定名称的文件。

程序首先调用 `open()` 打开指定目录，再通过 `fstat()` 判断当前路径对应的是普通文件还是目录。如果是普通文件，则直接比较文件名；如果是目录，则不断读取目录项，对每一个有效目录项拼接形成新的路径，再递归调用查找函数。

2.5 xargs

`xargs` 程序的功能是从标准输入读取内容，将读取到的字符串作为参数附加到原有命令之后，然后通过 `fork()` 和 `exec()` 执行命令。

实现时首先保存命令行中原有参数，再逐字符读取标准输入，将一行数据划分成不同参数。每读取完一行，就创建一个子进程，并调用 `exec(argv[1], args)` 执行对应命令。父进程通过 `wait()` 等待当前子进程执行结束，然后继续处理下一行输入。该实验综合使用了标准输入；字符串解析；`fork()`；`exec()`；`wait()`。

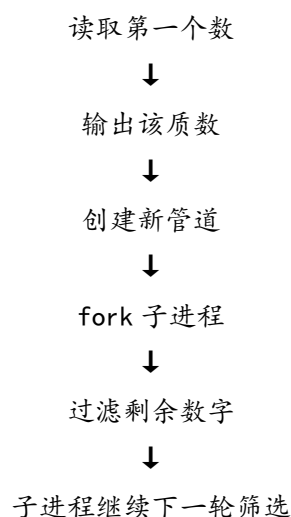
3. 实验过程中遇到的问题及解决

问题一：primes 中多个筛选进程如何组织，才能实现连续的质数筛选

在刚开始实现 `primes` 时，较难确定如何将教材中的素数筛算法转换成 `xv6` 的进程模型。如果由一个进程直接完成所有数字判断，虽然能够得到正确的质数结果，但并不符合实验要求，因为实验要求使用多个进程和管道组成流水线。

经过分析，每一个筛选阶段都可以看作一个独立进程：当前进程读取第一个数字并将其作为质数，再将剩余数字中过滤后的结果传给下一个进程。这样，每个进程只负责一个质数的筛选。

因此采用递归方式组织：



这种方式最终形成了多个通过管道串联的进程。

问题二：find 递归遍历目录时出现无限递归和路径处理错误

在实现 `find` 时，需要递归访问目录中的所有文件和子目录。如果对目录项不做特殊处理，会继续进入当前目录或上一级目录，最终造成无限递归。此外，`xv6` 对路径长度和目录

项名称长度均有限制，如果直接进行字符串拼接，也可能导致生成的路径超过缓冲区范围。

我的解决方法是在遍历目录项时首先判断目录项是否有效，然后显式跳过：.和..，并在拼接新路径前检查路径长度是否超过缓冲区大小。之后再当前目录路径、/ 和目录项名称进行组合，并递归调用 `find`。

4. 实验心得

作为 xv6 系列实验的第一个实验，Util 实验本身并没有修改大量内核代码，但它帮助我建立了对 xv6 开发模式的初步认识。

通过 `sleep` 程序，我熟悉了 xv6 用户程序的基本结构以及系统调用的基本使用方式。`pingpong` 和 `primes` 让我对 Unix 的进程和管道模型有了更加直观的认识。尤其是 `primes`，将一个素数筛选算法拆分为多个独立进程，每个进程通过管道接收和输出数据，使我真正体会到了 Unix 中“每个程序只做好一件事，再通过管道组合程序”的设计思想。`find` 实验使我开始接触 xv6 文件系统接口。`xargs` 则让我综合使用了输入解析、`fork()`、`exec()` 和 `wait()`。通过该程序，我进一步理解了 shell 启动程序的基本思想：首先组织参数，然后创建进程并通过 `exec()` 将当前进程替换为目标程序。

总体而言，本实验虽然主要集中于用户态，但已经涉及了后续 xv6 实验中大量重要概念，包括进程创建、进程间通信、程序加载、文件访问以及系统调用。通过完成这些 Unix 工具，我逐渐从使用操作系统提供的命令，转变为理解这些命令如何依靠操作系统机制实现，为后续深入修改 xv6 内核打下了良好的基础。

实验二：System Calls

1. 实验目的

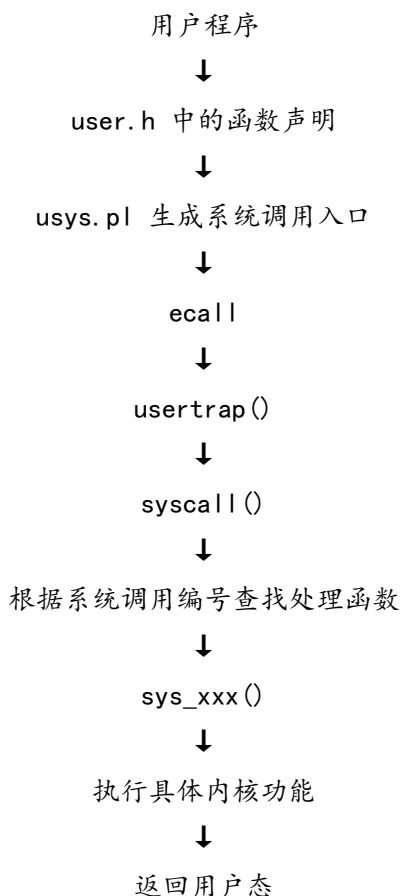
本实验主要通过通过在 xv6 中增加新的系统调用，理解用户程序进入内核并调用内核功能的完整过程，掌握系统调用编号、用户态接口、内核处理函数以及参数传递之间的关系。

实验主要实现 `trace` 和 `sysinfo` 两个系统调用。其中，`trace` 用于跟踪指定系统调用的执行情况，`sysinfo` 用于统计当前系统中的空闲内存字节数以及状态不为 `UNUSED` 的进程数量。

2. 实验内容

2.1 系统调用机制分析

在实现新的系统调用之前，首先对 xv6 原有的系统调用机制进行分析。用户程序调用一个系统调用时，并不是直接调用内核中的 `sys_xxx()` 函数，而是需要经过多个层次。整体过程可以概括为：



用户态的系统调用参数通常保存在 RISC-V 的寄存器中，其中系统调用编号保存在 `a7` 寄存器中，参数主要通过 `a0`、`a1` 等寄存器传递。进入内核后，`syscall()` 根据 `a7` 中保存的编号确定需要调用哪个系统调用处理函数，并将返回值重新写入 `a0`，最终返回用户程序。

通过分析这一流程，可以发现，新增一个系统调用通常需要同时修改多个文件，而不能只增加一个内核函数。

2.2 实现 trace 系统调用

trace 系统调用的功能是允许某个进程选择性地跟踪系统调用。当被跟踪进程调用指定系统调用时，内核输出该进程的 PID、系统调用名称以及返回值。

实现 trace 时，首先在进程结构体 struct proc 中增加一个变量，用于保存当前进程的系统调用跟踪掩码。该变量属于进程本身，因此不同进程可以拥有不同的跟踪设置。随后，新增 trace 系统调用的用户接口和系统调用编号，使用户程序能够调用：trace(mask)。进入内核后，在 sys_trace() 中通过 argint() 获取用户传入的掩码，并保存到当前进程结构体中。

接下来修改 syscall()。每次完成一个系统调用后，根据 $\text{mask} \& (1 \ll \text{syscall_number})$ 判断当前系统调用是否需要被跟踪。如果对应位为 1，则打印进程 PID + 系统调用名称 + 返回值。因此还需要建立系统调用编号和系统调用名称之间的对应关系。

2.3 处理 trace 在 fork 后的继承

实验要求通过 fork() 创建的新进程也应当继承父进程的跟踪掩码。因此仅在 sys_trace() 中设置当前进程的 mask 是不够的。如果子进程中的 mask 没有被初始化，使用 trace 启动一个会继续创建子进程的程序时，子进程就无法继续输出跟踪信息。因此需要在 fork() 创建子进程的过程中，将 $\text{np} \rightarrow \text{mask} = \text{p} \rightarrow \text{mask}$; 加入进程属性复制过程，这样就可以保证跟踪设置能够沿进程树继续传递。

2.4 实现 sysinfo 系统调用

sysinfo 系统调用用于获取系统当前运行状态，主要返回当前剩余的空闲内存字节数和当前处于使用状态的进程数量。

根据实验要求，用户程序调用 sysinfo(&info) 后，内核需要统计相关数据，并将结构体内容复制到用户空间。因此 sysinfo 的实现主要分为三个部分：统计空闲内存、统计进程数量、构造 struct sysinfo 并将数据返回到用户空间。

首先是统计空闲内存。xv6 使用空闲链表管理未分配的物理页。在 kalloc.c 中，空闲页被组织在：kmem.freelist 链表中。因此可以通过遍历该链表统计当前空闲页面数量。在遍历空闲链表时需要使用自旋锁进行保护，以避免多核环境下其他 CPU 同时执行 kalloc() 或 kfree()，造成统计过程中链表发生变化。

接着是统计系统进程数量。xv6 中所有进程通过固定大小的进程表进行管理。因此可以遍历整个 proc 数组，判断进程状态是否为 UNUSED。只要进程状态不是 UNUSED，就说明该进程槽当前已经被系统使用。

最后要将 sysinfo 数据返回用户空间。由于用户程序传递进来的地址属于用户虚拟地址，内核不能简单地直接通过指针进行写入，而需要使用 copyout() 将内核中的数据安全复制到对应用户页表中。

3. 实验过程中遇到的问题及解决

问题一：新增系统调用后用户程序无法正确进入对应的内核处理函数

在最初增加 trace 和 sysinfo 时，容易出现用户态已经写好了调用代码，但编译失败，或者程序执行后没有进入预期内核函数的问题。

经过分析发现，xv6 中一个完整的系统调用并不是只需要编写 sys_xxx() 函数，而是涉及多个文件之间的配合。新增系统调用通常需要完成以下文件的配置：user/user.h、

user/usys.pl、kernel/syscall.h、kernel/syscall.c、kernel/sysproc.c。其中任何一处遗漏，都可能导致编译错误或者系统调用编号无法正确匹配。

问题二：统计空闲内存时直接遍历 freelist 存在并发安全问题

实现 sysinfo 时，需要遍历 kmem.freelist 统计空闲物理页数量。如果直接进行 `for(r = kmem.freelist; r; r = r->next)`，虽然在单核或运行负载较低时可能得到正确结果，但在多核环境中，另一个 CPU 可能同时修改空闲链表。这样不仅可能导致统计结果不准确，还可能在遍历过程中读取到发生变化的链表节点。

因此，我的解决方法是：加锁。在统计前：`acquire(&kmem.lock)`；遍历结束后：`release(&kmem.lock)`。利用与内存分配器相同的锁保证统计操作期间链表结构保持一致。

4. 实验心得

System Calls 实验是我第一次真正对 xv6 内核的用户态与内核态接口进行修改。相比第一个 Util 实验主要使用已有系统调用，本实验需要完整地增加新的系统调用，因此让我对 xv6 的整体运行机制有了更加深入的认识。

通过 trace 实验，我理解了系统调用从用户程序执行到内核处理的完整过程。过去在 C 程序中调用 `read()`、`write()` 等函数时，通常只将它们当作普通接口，而本实验让我看到这些接口最终会通过 RISC-V 的 `ecall` 指令进入内核，由 `syscall()` 根据编号进行分发。系统调用编号实际上构成了用户程序和操作系统内核之间的重要协议。

通过 sysinfo 实验，我第一次跨越多个内核模块获取系统运行信息。一方面需要读取物理内存分配器的空闲链表，另一方面需要遍历进程表，还需要利用锁保证这些共享内核数据结构在多核环境下访问安全。因此，该实验不仅涉及系统调用，也让我提前接触了物理内存管理、进程管理和并发同步等后续实验的重要内容。

完成本实验后，我不再只是从用户程序角度使用操作系统接口，而是能够从内核角度理解一个系统调用是如何被定义、执行并返回结果的，为后续实验打下了重要基础。

实验三：Page Tables

1. 实验目的

本实验主要通过修改 xv6 的页表管理机制，加深对 RISC-V Sv39 三级页表结构、虚拟地址到物理地址转换过程以及内核页表 and 用户页表之间关系的理解。

实验按照教程主要完成三个任务：首先实现页表打印函数 `vmprint()`，通过递归遍历页表观察三级页表的实际组织方式；其次为每个进程建立独立的内核页表，并在进程调度时切换相应页表；最后将用户地址空间映射到进程自己的内核页表中，使用新的 `copyin` 和 `copyinstr` 实现，使内核能够更加直接地访问用户空间数据。

通过本实验，可以进一步掌握页表创建、映射、切换和释放过程，同时理解页表在进程隔离和用户态、内核态地址转换中的作用。

2. 实验内容

2.1 实现页表打印功能

实验的第一个任务是实现 `vmprint()` 函数，用于打印指定页表中的有效页表项。官方要求在第一个用户进程完成 `exec()` 后打印其页表，从而观察实际运行中的 Sv39 三级页表结构。

RISC-V Sv39 采用三级页表，每一级页表包含 512 个页表项。实现时从最高级页表开始遍历，对所有设置了 `PTE_V` 标志的有效页表项进行输出。如果当前页表项仍然指向下一级页表，则继续递归遍历，并根据当前层级增加缩进，以直观表示三级页表的树形结构。

每个有效页表项需要输出其页表项内容以及对应的物理地址。通过页表打印结果，可以看到用户程序代码、数据、栈、trampoline 等区域最终通过多级页表映射到不同的物理页面。完成该部分后，在 `exec()` 结束前对第一个进程调用 `vmprint()`，并通过官方测试检查输出格式是否正确。

2.2 为每个进程建立独立的内核页表

原始 xv6 中所有进程进入内核后共同使用全局 `kernel_pagetable`。本实验第二部分要求修改这一机制，使每个进程都拥有自己的内核页表，并在调度不同进程时切换到对应的内核页表。

首先在 `struct proc` 中增加保存进程内核页表的成员。随后参考原有 `kvminit()` 的实现，为每个新创建的进程单独建立一份内核页表，其中仍然包含 UART、VirtIO、PLIC、内核代码、内核数据以及 trampoline 等必要的内核映射。由于每个进程都有独立的内核栈，因此还需要将该进程自己的 `kernel stack` 映射到对应的内核页表中，而不能继续完全依赖原来的全局内核栈映射。

完成页表创建后修改 `scheduler()`。当调度器准备运行某个进程时，将该进程的内核页表地址写入 `satp` 寄存器，并通过 `sfence_vma()` 刷新 TLB；当 CPU 没有运行任何用户进程、重新回到调度器时，则切换回原来的全局 `kernel_pagetable`。

进程退出时，还需要在 `freeproc()` 中释放为该进程建立的内核页表。由于这些页表中的部分映射只是引用已经存在的物理内存，因此释放时需要只释放页表本身，而不能将被映射的物理页面一起释放，否则会破坏内核原有的内存区域。

这一部分完成后，通过运行 `usertests` 检查进程创建、调度、退出以及文件系统等原有功能是否仍能够正常工作。

2.3 简化 `copyin` 和 `copyinstr`

实验最后一个任务是在每个进程的内核页表中加入该进程用户空间的映射，使内核能够使用与用户程序相同的虚拟地址访问用户内存，并利用实验提供的 `copyin_new()` 和 `copyinstr_new()` 代替原来的软件页表遍历方式。

原来的 `copyin()` 在需要读取用户空间数据时，会使用用户页表将用户虚拟地址转换为物理地址，再通过内核的直接映射访问该物理内存。新的方案则将用户空间中的相关页面同步映射到每个进程自己的内核页表中，这样在内核态访问用户地址时，就可以直接通过硬件页表完成地址转换。

为实现这一功能，需要保证用户页表发生变化时，进程内核页表中的对应映射也同步变化。因此，在 `fork()`、`exec()`、`sbrk()` 以及第一个进程初始化等涉及用户地址空间创建、扩展或替换的位置，都需要维护对应的内核页表映射。官方教程同时要求限制用户进程的最大地址不能超过 PLIC 所在地址，以避免用户地址区域和内核已有虚拟地址区域发生冲突。

在建立用户空间到内核页表的映射时，还需要特别处理页表权限。用户页表中的页面通常带有 `PTE_U` 权限，但 RISC-V 在 Supervisor 模式下不能按照这种权限直接访问，因此映射到进程内核页表后需要去掉 `PTE_U` 标志，同时保留相应的读写执行权限。

完成映射同步后，将原有 `copyin()` 替换为实验提供的 `copyin_new()`，再以相同方式替换 `copyinstr()`。最后运行 `usertests` 和 `make grade`，检查页表映射以及原有用户程序功能是否正常。

3. 实验过程中遇到的问题及解决

问题一：将用户页面映射到进程内核页表后，`copyin` 仍然不能直接访问用户地址

在完成用户页面向进程内核页表的映射后，如果直接复制用户页表中的权限标志，新的 `copyin_new()` 仍可能无法正常读取用户空间数据。

分析 RISC-V 页表权限后发现，用户页面通常设置有 `PTE_U` 标志，该标志表示该页面可以在 User 模式下访问。将同样的页表项直接放入进程内核页表后，Supervisor 模式下并不能按照实验要求直接访问这些映射。

因此，在向进程内核页表复制用户页面映射时，需要去掉 `PTE_U` 权限，同时保留页面原本需要的读、写或执行权限。

修改页表项权限之后，`copyin_new()` 和 `copyinstr_new()` 即可利用进程内核页表直接访问相应的用户虚拟地址。

问题二：用户页表发生变化后，进程内核页表中的映射没有同步更新

在实现新的 `copyin` 机制时，仅在创建进程内核页表时复制一次用户地址空间是不够的。进程运行过程中，其用户页表还会不断发生变化，如果只修改用户页表，而没有同步修改进程内核页表，两套页表中同一个虚拟地址就可能对应不同的映射，最终导致 `copyin_new()` 访问错误地址或者触发 `page fault`。

我的解决方法是在所有可能修改用户空间映射的位置同步维护进程内核页表。对于增加的用户页面，在内核页表中增加相同物理页面的映射；对于已经释放的用户页面，同时删除

内核页表中的对应映射；对于 `exec()` 重新创建地址空间的情况，则根据新的用户页表重新建立对应关系。

4. 实验心得

Page Tables 实验是我第一次较大范围修改 xv6 的虚拟内存管理机制。相比前两个实验主要集中在用户程序和系统调用接口，本实验已经深入到操作系统最核心的地址空间管理部分，对代码正确性的要求也明显提高。

通过实现 `vmprint()`，我对 RISC-V Sv39 三级页表结构有了更加直观的认识。过去学习页表时主要通过课本中的示意图理解 VPN、PTE 和物理页之间的关系，而实际递归打印 xv6 页表之后，可以看到三级页表真正以树形结构组织，并能够结合页表项直接分析某个虚拟地址最终如何映射到物理内存。

为每个进程建立独立内核页表的过程，则让我进一步理解了页表与进程上下文之间的关系。进程切换不仅需要切换寄存器和运行状态，在本实验修改后的结构中还需要切换 `satp`，使 CPU 使用当前进程对应的内核页表。同时还必须利用 `sfence_vma()` 刷新 TLB，保证新的地址映射能够立即生效。这让我认识到进程调度、CPU 状态和虚拟内存之间并不是完全独立的模块，而是存在紧密联系。

最后，通过修改 `copyin` 和 `copyinstr`，我理解了 xv6 原有的用户空间数据访问方式以及利用页表硬件简化地址转换的方法。为了实现这一目标，需要同时维护用户页表和进程内核页表，并处理 `fork()`、`exec()` 和 `sbrk()` 等多个内存变化位置。这一过程使我更加深刻地理解了操作系统维护虚拟地址空间一致性的复杂性。

总体而言，本实验使我对虚拟内存从理论上的“地址转换机制”进一步深入到操作系统中的“页表创建、映射、切换、同步和释放”全过程，为后续 Lazy Allocation、Copy-on-Write 和 `mmap` 等与虚拟内存密切相关的实验打下了重要基础。

实验四：Traps

1. 实验目的

本实验主要通过分析 RISC-V 汇编、实现内核栈回溯以及实现周期性用户态 Alarm 机制，进一步理解 xv6 中的 Trap 处理流程以及用户态和内核态之间的切换过程。

实验按照教程主要分为三个部分：首先通过阅读 RISC-V 汇编代码理解函数调用约定、寄存器使用和栈帧结构；其次实现 `backtrace()` 函数，通过保存的 `frame pointer` 逐层回溯内核函数调用栈；最后实现 `sigalarm` 和 `sigreturn` 系统调用，使用户进程能够按照设定的时间间隔执行用户态处理函数，并在处理完成后恢复到被中断的位置继续运行。

通过本实验，可以进一步理解 `trapframe`、`sepc`、时钟中断、寄存器保存与恢复等机制，以及 Trap 在系统调用、中断和异常处理中的重要作用。

2. 实验内容

2.1 分析 RISC-V 汇编与函数调用过程

实验的第一部分要求阅读 `user/call.c` 编译后生成的 `user/call.asm`，结合 RISC-V 函数调用约定分析函数参数传递、返回地址保存、函数调用以及大小端等问题，并将结果写入 `answers-traps.txt`。

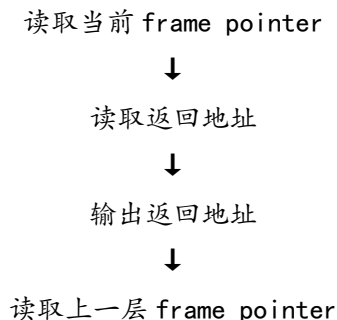
通过分析汇编代码，可以看到 RISC-V 主要通过 `a0~a7` 寄存器传递函数参数，其中 `a0` 和 `a1` 也常用于保存返回值；`ra` 寄存器用于保存函数调用后的返回地址；`sp` 表示当前栈指针，而 `s0` 在编译器生成的代码中通常作为 `frame pointer` 使用。

实验过程中还需要分析 `f()` 和 `g()` 等函数的调用。由于编译器会对简单函数进行内联优化，因此在最终生成的汇编代码中不一定能够找到明确的函数调用指令。这使我进一步认识到 C 语言源程序和最终机器指令之间并不一定是一一对应关系。

2.2 实现内核栈回溯 `backtrace`

实验的第二部分要求实现 `backtrace()` 函数，使内核能够在运行过程中输出当前函数的调用链，从而辅助内核调试。

实现过程中利用 RISC-V 的函数栈帧结构进行回溯。编译器将当前函数的 `frame pointer` 保存在 `s0` 寄存器中，因此首先在 `riscv.h` 中实现读取 `s0` 的辅助函数 `r_fp()`，得到当前栈帧地址。根据 RISC-V 函数调用约定，一个栈帧中保存着当前函数返回到上一级函数所需要的信息。其中返回地址位于 `frame pointer` 之前固定的位置，上一层函数的 `frame pointer` 也保存在固定偏移处。因此可以从当前 `frame pointer` 开始：





继续回溯

由于 xv6 为每个内核栈分配一个页面，因此可以利用 `PGROUNDDOWN()` 和 `PGROUNDUP()` 确定当前内核栈的有效范围，当 `frame pointer` 超出当前栈页后停止回溯，避免访问非法地址。完成后使用 `addr2line` 工具将输出的返回地址转换为对应源码文件和代码行，从而验证函数调用关系是否正确。

2.3 实现 Alarm 机制

实验的最后一个部分是在 xv6 中增加用户态 Alarm 机制。用户程序通过 `sigalarm(interval, handler)` 设置 Alarm，当该进程累计运行指定数量的时钟 tick 后，内核将用户程序的执行位置暂时切换到 `handler` 函数。处理函数执行结束后通过 `sigreturn()` 恢复被时钟中断打断之前的用户程序状态，并继续原来的程序。

首先要增加 `sigalarm` 和 `sigreturn` 两个系统调用，并在 `struct proc` 中增加 Alarm 相关信息，包括 Alarm 间隔、处理函数地址、当前经过的 tick 数量，以及保存被中断用户上下文所需要的数据。

随后修改 `usertrap()`。每次发生时钟中断时判断当前进程是否启用了 Alarm，并更新计数。当累计运行时间达到设定值时，将当前用户态执行现场保存下来，并修改 `trapframe` 中的程序计数器，使 Trap 返回用户态后不再从原来的程序位置继续执行，而是转而执行用户指定的 `handler`。

仅修改执行地址虽然可以进入 `handler`，但 `handler` 结束后还需要恢复原程序，因此还需要实现 `sigreturn()`。在 Alarm 触发前保存完整的用户寄存器状态，当用户处理函数调用 `sigreturn()` 时，将原先保存的 `trapframe` 重新恢复，使用户进程继续执行被中断之前的代码。

此外，还需要防止 Alarm 处理函数发生重入。如果 `handler` 尚未执行结束，即使又发生了新的时钟中断，也不能再次进入 `handler`，否则前一次保存的执行现场会被覆盖。因此在进程结构中增加状态标记，只允许当前 Alarm 处理结束并执行 `sigreturn()` 之后重新启用下一次 Alarm。

3. 实验过程中遇到的问题及解决

问题一：backtrace 函数如果没有正确设置终止条件，会访问当前内核栈之外的非法地址

实现 `backtrace()` 时需要沿着 `frame pointer` 逐级访问上一层栈帧。如果只根据上一层 `frame pointer` 是否为 0 作为结束条件，无法保证回溯过程中始终处于当前进程的有效内核栈范围内。一旦读取到异常的 `frame pointer`，继续根据固定偏移访问返回地址，就可能访问当前内核栈页面之外的地址，甚至引发新的 `kernel panic`。

经过分析，xv6 为每个内核栈分配一页内存，因此可以根据初始 `frame pointer` 计算当前内核栈的上下边界，并确保后续 `frame pointer` 始终位于该页面范围内。实现时利用 `PGROUNDDOWN(fp)` 确定栈页面下边界，并利用 `PGROUNDUP(fp)` 确定上边界，当 `frame pointer` 离开这一有效范围时立即停止回溯。

问题二：Alarm 处理函数能够被调用，但返回后用户程序无法继续正常运行

Alarm 功能实现过程中，一个重要问题是仅修改 `trapframe->epc` 虽然能够使用户程序跳转到 handler，但 handler 执行完成以后无法自动恢复到原程序被中断的位置。

经过分析，我发现这是因为时钟中断发生时，用户程序不仅有一个当前执行地址，还具有大量正在使用的寄存器状态。如果只保存原来的 `epc` 而没有完整保存寄存器，Alarm 处理函数本身会改变 `a0`、`ra`、`sp` 以及其他寄存器内容。当 handler 结束以后，即使恢复到原来的程序地址，程序上下文也已经发生改变，从而可能出现 `alarmtest` 无法继续执行或测试失败的问题。官方 `alarmtest` 的后续测试正是要求恢复被中断代码，并恢复中断发生时的寄存器内容。

我在解决时，不再只保存单独的程序计数器，而是在 Alarm 触发时保存完整的用户 `trapframe`。当 handler 最终调用 `sigreturn()` 时，再将保存的 `trapframe` 内容恢复到当前进程的 `trapframe` 中。

4. 实验心得

Traps 实验让我更加深入地理解了用户程序、CPU 和操作系统内核之间是如何通过 Trap 机制联系起来的。相比之前 System Calls 实验主要从系统调用接口角度理解用户态进入内核态，本实验进一步深入到寄存器、栈帧、中断和执行现场恢复等底层机制。

通过 RISC-V 汇编分析，我进一步熟悉了函数参数传递、`ra` 返回地址、`sp` 栈指针以及 `s0` 作为 `frame pointer` 的作用。同时也认识到编译器可能进行函数内联等优化，因此高级语言源码并不一定与汇编指令逐行对应。这部分内容为后续理解内核栈和 Trap 执行过程提供了基础。

实现 `backtrace()` 后，我对函数调用栈的认识从理论概念变得更加具体。每一次函数调用都会留下能够恢复上一级函数执行状态的信息，而 `frame pointer` 将不同栈帧串联起来。利用这些信息，操作系统可以在发生 `panic` 时输出完整调用链，这也是实际操作系统调试中非常重要的方法。

Alarm 部分是本实验最核心的内容。实现 Alarm 不仅需要处理时钟中断，还需要修改用户程序的执行位置、保存完整 `trapframe`、恢复寄存器状态并避免处理函数重入。通过这一过程，我更加深刻地理解了 `usertrap()` 和 `usertrapret()` 所承担的作用，也理解了为什么 Trap 发生时必须保存完整的用户态执行现场。

尤其是在实现 `sigreturn()` 后，我认识到中断处理的关键并不是简单地“暂停一个程序并运行另一个函数”，而是必须做到处理中断之后能够让原来的程序感觉自己从未被打断过。这要求操作系统准确保存和恢复程序计数器、通用寄存器以及其他关键状态。

实验五：Lazy Page Allocation

1. 实验目的

本实验主要通过通过在 xv6 中实现用户堆内存的惰性分配机制，加深对虚拟内存、页表、缺页异常以及物理内存分配之间关系的理解。

在原始 xv6 中，用户程序调用 `sbrk()` 扩展地址空间时，内核会立即分配对应的物理页面并建立页表映射。当程序一次申请大量内存时，即使其中部分内存最终并没有被实际访问，系统仍然需要提前完成所有物理页的分配和映射，造成不必要的时间和空间开销。

Lazy Allocation 的基本思想是：调用 `sbrk()` 时只扩大进程所拥有的虚拟地址空间范围，而不立即分配物理内存。当程序第一次真正访问某个尚未分配的页面时，由 CPU 产生 `page fault`，内核再为该虚拟页面分配物理内存并建立映射。

通过本实验，主要掌握 `sbrk()` 的内存扩展机制、RISC-V 的 `page fault` 处理、用户页表的动态建立以及异常情况下的地址合法性检查，并为后续 Copy-on-Write 和 `mmap` 实验打下基础。

2. 实验内容

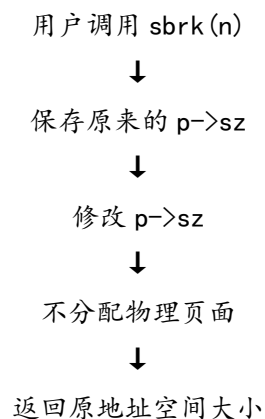
2.1 取消 `sbrk` 中的立即物理内存分配

实验首先要求修改 `sys_sbrk()`，取消原有的立即物理内存分配行为。

原始 xv6 中，`sys_sbrk()` 通过 `growproc()` 扩大进程地址空间，而 `growproc()` 进一步调用 `uvmmalloc()` 分配物理页面并建立页表映射。因此，每当用户程序调用 `sbrk(n)`，内核都会立即为新增的虚拟地址范围准备物理内存。

惰性分配要求修改这一过程，使 `sbrk()` 只更新进程的地址空间大小 `p->sz`，并返回扩展之前的旧地址，而不调用 `growproc()` 完成物理页分配。官方实验明确要求这一阶段只增加进程地址空间大小，不实际分配页面。

修改后的基本过程为：



完成这一修改后，用户程序虽然在逻辑上已经拥有新增的虚拟地址范围，但页表中还没有这些地址对应的有效页表项。因此，当程序真正访问这些地址时，CPU 会产生 `page fault`。这正是后续实现 Lazy Allocation 的基础。

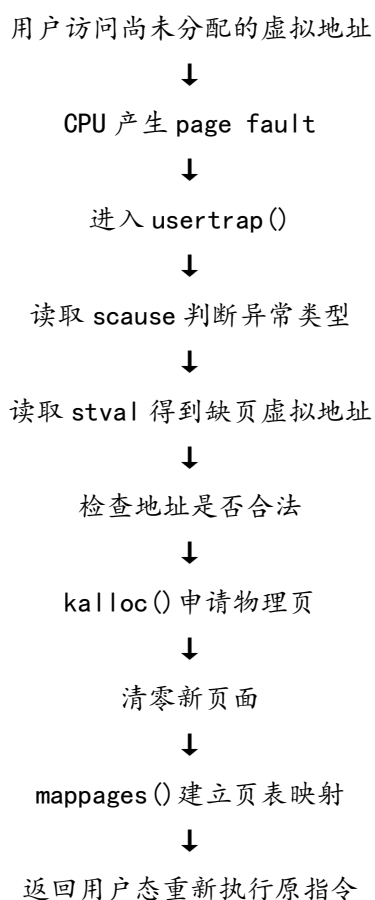
2.2 处理 page fault 并实现 Lazy Allocation

取消 `sbrk()` 中的物理内存分配后，程序第一次访问新增内存时会产生 `page fault`。如果内核仍然按照原来的 `Trap` 逻辑处理，就会将其视为无法识别的用户异常并终止进程。

因此实验的第二个主要任务是在 `usertrap()` 中识别并处理由惰性分配产生的 `page fault`。

RISC-V 可以通过 `scause` 判断异常类型，其中用户程序发生 `load page fault` 或 `store page fault` 时，对应的 `scause` 值分别为 13 和 15。发生 `page fault` 后，可以通过 `stval` 寄存器获得引发异常的虚拟地址。官方要求利用 `r_scause()` 和 `r_stval()` 获得这些信息，并参考 `uvmalloc()` 中的代码为缺页地址分配物理页。

处理过程主要为：



由于 `stval` 记录的地址不一定正好位于页面起始位置，因此需要使用 `PGROUNDDOWN()` 将地址向下取整到页边界，再建立相应页表映射。同时，新申请的物理页需要使用 `memset()` 清零，保证用户程序第一次访问新分配内存时得到的内容符合正常内存分配语义。

由于采用惰性分配后，一个进程在 `0~p->sz` 之间可能存在合法但尚未实际映射的页面，因此原有 `uvmunmap()` 中“页面不存在就 `panic`”的处理方式也不再适用。释放地址空间时如果遇到一个合法但从未被实际访问、因而没有分配物理页的虚拟页面，应直接跳过，而不是触发 `kernel panic`。官方教程也明确要求修改 `uvmunmap()` 以允许存在未映射页面。

完成这一部分后，普通用户程序可以正常使用惰性分配的内存，例如 `shell` 执行 `echo hi` 时，即使内部调用 `sbrk()` 得到的内存最初并未映射，也可以在首次访问时由 `page fault` 处理程序自动完成分配。

2.3 完善 Lazy Allocation

完成基本 page fault 处理后，还需要处理一些特殊情况，使惰性分配机制能够在完整的 xv6 运行环境中正常工作。

首先需要正确处理负数 sbrk()。当用户通过 sbrk() 缩小地址空间时，已经实际分配的物理页面仍然需要正常释放，因此不能简单地只减少 p->sz，而应调用相应的页表释放逻辑回收已映射页面。

其次，page fault 处理程序必须判断缺页地址是否属于合法的进程地址空间。如果用户访问的地址超过 sbrk() 已经申请的范围，或者落入用户栈下方的无效保护区域，就不能进行 Lazy Allocation，而应该将当前进程标记为终止。

fork() 同样需要进行修改。原有 uvmcopy() 默认父进程从 0 到 p->sz 之间的所有页面都已经存在，因此遇到未映射页面时可能产生错误。而惰性分配允许这些区域中存在尚未实际使用的页面，因此复制父进程地址空间时，如果发现某个页面没有对应的 PTE，应直接跳过，而不是将其视为异常。

此外，如果用户将一个由 sbrk() 获得但还没有实际访问的地址直接传给 read() 或 write() 等系统调用，page fault 不会一定发生在用户态，而可能是内核在执行 copyin() 或 copyout() 时发现对应页面不存在。因此需要保证这类合法的惰性页面同样能够被正确分配，从而使系统调用能够正常完成。

最后还需要处理物理内存不足的情况。如果 page fault 发生后调用 kalloc() 无法得到新的物理页，则说明系统已经没有足够内存继续完成该进程的请求，此时应终止当前进程，而不能继续使用无效物理地址。

3. 实验过程中遇到的问题及解决

问题一：fork 复制父进程地址空间时，遇到尚未分配的惰性页面会产生异常

实现基本 Lazy Allocation 后，普通程序已经可以通过 page fault 完成页面分配，但在创建子进程时仍出现异常。经分析发现，原因是原有 uvmcopy() 假设父进程从 0 到 p->sz 之间的虚拟页面全部已经建立映射。因此它会逐页调用 walk() 检查父进程 PTE，如果发现某一页没有映射，则将其视为严重错误。

但引入 Lazy Allocation 以后，0~p->sz 只表示用户程序已经申请了这一范围的虚拟地址，并不代表其中每一个页面都已经分配物理内存。例如父进程申请了多个页面，但只实际访问其中一个页面，则剩余页面没有有效 PTE 是完全正常的。

因此，我的解决方案是修改 uvmcopy()。如果遍历过程中发现某个页面尚未映射，应跳过该页面，而不是执行 panic。对于已经实际分配的页面，则仍按照原有逻辑为子进程分配物理页并复制内容。

这样可以实现子进程继承的实际上是：父进程的地址空间大小、父进程已经实际分配的页面和其余尚未使用的页面继续保持惰性分配状态。

问题二：page fault 处理程序如果只判断地址是否小于 `p->sz`，会错误分配用户栈保护区

在处理 page fault 时，最基本的合法性判断是缺页地址不能超过当前进程的 `p->sz`。但经实验发现，仅依靠这一条件仍然不够。

xv6 的用户栈附近存在一个不可访问的保护页面，用于检测栈向非法区域扩展。如果用户访问该保护页面，正确行为应该是终止进程，而不是为它动态分配物理内存。如果 page fault 处理代码只判断 `va < p->sz`，就可能错误地将保护页视为普通的惰性页面，并为其建立映射，从而破坏原有的栈保护机制。因此需要结合用户栈位置进一步检查缺页地址。如果地址位于用户栈下方的无效区域，则直接将进程标记为 `killed`，而不能进入正常的 Lazy Allocation 流程。官方测试也专门检查了这一情况。

4. 实验心得

Lazy Allocation 实验让我第一次完整地从 page fault 角度修改 xv6 的虚拟内存行为。与 Page Tables 实验主要关注页表结构和映射方式不同，本实验更加关注什么时候分配物理内存这一问题，使我进一步理解了虚拟内存不仅是一种地址转换机制，同时也是操作系统进行资源管理和性能优化的重要手段。

在原始 xv6 中，调用 `sbrk()` 以后立即分配全部物理页面，整个流程相对简单。而 Lazy Allocation 将“申请虚拟地址空间”和“真正获得物理内存”两个过程分离开来。程序可以先拥有一段合法的虚拟地址范围，只有在真正访问某一页时才为其分配实际内存。这种机制可以减少不必要的物理页分配，同时缩短大规模内存申请时的处理时间。

通过修改 `usertrap()` 处理 page fault，我也进一步巩固了上一实验中学习的 Trap 机制。上一实验主要处理中断和 Alarm，而本实验中 page fault 同样通过 Trap 进入内核。内核需要读取 `scause` 判断异常类型，并通过 `stval` 获得引发异常的虚拟地址，再结合进程当前地址空间判断该异常究竟属于合法缺页还是非法访问。

实验中另一个重要收获是认识到修改内存分配策略会对多个内核模块产生影响。最初看起来只需要修改 `sbrk()` 和 page fault 处理，但进一步测试后还必须修改 `uvmunmap()`、`uvmcopy()` 以及系统调用访问用户地址的相关逻辑。其原因在于原始 xv6 很多代码都隐含了一个假设，即 `0~p->sz` 之间的用户页面已经全部存在；而 Lazy Allocation 打破了这一假设。因此，本实验让我更加深刻地认识到操作系统各模块之间存在较强关联。修改一个基础机制后，需要重新检查其他代码是否仍然满足原来的前提条件，而不能只保证局部代码正确。

实验六：Copy-on-Write Fork

1. 实验目的

本实验主要通过通过在 xv6 中实现 Copy-on-Write Fork 机制，加深对 fork()、用户页表、page fault、页表权限以及物理内存管理机制的理解。

原始 xv6 在执行 fork() 时，会调用 uvmcopy() 为子进程重新分配物理页面，并将父进程的用户内存逐页复制到子进程中。虽然这种方式实现简单，但复制过程开销较大。实际上，很多子进程在 fork() 之后很快就会调用 exec() 替换整个地址空间，此时之前复制的大量内存实际上从未被使用。

Copy-on-Write 的基本思想是：fork() 时不立即复制父进程的物理页面，而是让父子进程暂时共享相同的物理页，并将这些页面设置为只读。当父进程或子进程尝试写入共享页面时，CPU 产生 page fault，内核再为当前进程分配新的物理页面并复制原页面内容，从而实现“只有真正发生写操作时才复制”。

通过本实验，主要掌握 COW 页表映射、写时复制异常处理以及物理页面引用计数等机制，并进一步理解虚拟内存存在减少内存占用和提高进程创建效率方面的作用。

2. 实验内容

2.1 修改 fork，实现父子进程共享物理页面

实验首先需要修改 uvmcopy()，改变原有 fork() 复制整个用户地址空间的方式。

原始的实现会遍历父进程的用户页表，为每一个页面调用 kalloc() 申请新的物理页，再通过 memmove() 复制页面内容，最后建立子进程页表映射。实现 COW 后，不再为子进程立即分配新的物理页，而是让父进程和子进程的页表项同时指向同一个物理页面。

为了防止父子进程直接修改同一个页面，需要将原本具有写权限的用户页面清除 PTE_W 权限，使其暂时变为只读。同时，需要在页表项的软件保留位中选择一个标志位，用于区分本身就是只读的页面和因 COW 机制而临时改成只读的页面。

完成这一修改后，fork() 创建子进程时不再复制全部物理内存，从而大幅降低进程创建过程中的内存和时间开销。

2.2 处理 COW 写 page fault

父子进程共享页面后，共享页已经被设置为只读，因此一旦用户程序尝试写入这些页面，RISC-V 处理器就会产生 store page fault。因此实验的第二个核心任务是在用户 Trap 处理中识别 COW 页面产生的写异常，并完成真正的页面复制。

发生 page fault 后，首先通过 r_scause() 判断是否属于 store page fault，再利用 r_stval() 获得引发异常的用户虚拟地址。随后检查该地址对应的 PTE 是否满足 COW 条件：（1）页表项有效；（2）属于用户页面；（3）当前没有写权限；（4）设置了 COW 标志。只有满足这些条件时才进行 Copy-on-Write 处理。

对于需要复制的页面，首先使用 kalloc() 申请一个新的物理页，然后使用原物理页中的数据初始化新页面，再修改当前进程的页表项，使其指向新的物理地址，并恢复 PTE_W 写权限，同时取消 COW 标志。这样，真正执行写操作的进程获得了自己的独立物理页面，而

另一个进程仍然保留原来的页面，因此父子进程之间的内存隔离得到恢复。

除用户程序直接写内存外，内核中的 `copyout()` 也可能向用户空间写数据。如果目标页面属于 COW 页面，同样需要完成写时复制。因此还需要修改 `copyout()` 相关逻辑，使内核向 COW 用户页面写数据时也能够先创建私有副本，而不是因为页面没有写权限而直接失败。

2.3 实现物理页面引用计数

COW 机制使一个物理页面可能同时被多个进程的页表引用，因此原有的 `kfree()` 逻辑已经不能继续直接使用。

原始 xv6 中，一个页面只属于一个使用者，只要解除对应映射就可以调用 `kfree()` 将该页面放回空闲链表。但在 COW 情况下，如果父进程和子进程都指向同一个物理页面，其中一个进程退出时立即释放该物理页，就会导致另一个仍在使用该页面的进程访问已经释放的内存。因此需要为每个物理页增加引用计数。

当 `kalloc()` 分配一个新的物理页面时，将其引用计数初始化为 1；当 `fork()` 通过 COW 方式让子进程共享父进程的物理页时，通过类似 `krefinc(pa)` 的函数增加该页面的引用次数。当某个进程不再使用该页面时调用 `kfree()`，此时不再无条件释放页面，而是首先减少引用计数。

由于引用计数本身可能被多个 CPU 同时修改，因此还需要使用自旋锁保护引用计数数组，避免并发执行 `fork()`、`exit()` 或 `page fault` 处理时产生竞争。

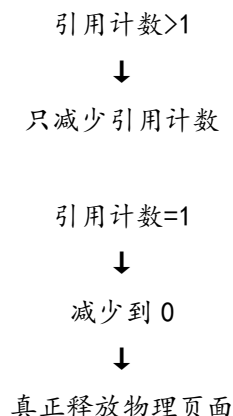
完成引用计数机制后，父子进程可以安全共享同一个物理页面，只有最后一个使用者释放该页时，物理页面才真正返回空闲链表。

3. 实验过程中遇到的问题及解决

问题一：实现 COW 后，如果 `kfree` 仍直接释放物理页面，会导致共享页面被提前回收

在完成 `uvmcopy()` 的 COW 修改后，父子进程已经开始共享相同的物理页面。此时如果仍然使用原始的 `kfree()` 实现，当其中一个进程退出、缩小地址空间或发生页表释放时，就会直接将共享的物理页加入 `kmem.freelist`。但该物理页面可能仍然被另一个进程的页表引用。如果该页面被重新分配给其他用途，剩余进程继续通过旧 PTE 访问它，就可能读取或修改完全错误的数据，造成内存破坏，严重时甚至导致测试异常或 `kernel panic`。

因此，我需要重新设计 `kfree()`，使页面是否真正释放由引用计数决定，而不能由一次调用 `kfree()` 直接决定。修改后的核心思想是：



问题二：krefinc 和 kfree 中的引用计数更新如果缺少锁保护，会产生并发错误

实验过程中需要实现 krefinc(), 其基本功能是增加某个物理页面的引用计数。在修改这一部分时, 需要特别考虑多核情况下引用计数更新的原子性。例如两个 CPU 可能同时执行 fork() 并增加同一个物理页面的引用次数, 也可能一个 CPU 正在增加引用计数, 而另一个 CPU 正在通过 kfree() 减少引用计数。如果没有锁保护, 则这些读、修改、写操作可能交叉执行, 从而导致计数结果与真实引用数量不一致。

因此, 我为引用计数结构增加了独立的自旋锁, 在每次读取或更新计数时进行保护。同时需要避免在持有引用计数锁的情况下进行不必要的复杂操作。判断计数下降到 0 之后, 可以再按照正确的锁顺序将页面加入 freelist, 从而避免不同锁之间形成错误的嵌套关系。

问题三：物理地址到引用计数数组下标的转换必须保持一致

实现引用计数时, 需要为每一个可分配物理页面维护一个计数值。实验过程中如果直接将物理地址作为数组下标, 不仅会造成数组空间巨大, 还容易发生越界访问。因此需要按照页面大小将物理地址转换为页编号。

同时, kalloc()、kfree() 和 krefinc() 必须使用完全一致的转换方式。如果不同函数对下标的计算方式不同, 就会出现“增加的是一个位置, 减少的是另一个位置”的错误, 最终导致引用计数失效。

4. 实验心得

Copy-on-Write Fork 实验让我对 fork() 和虚拟内存之间的关系有了更加深入的认识。原始 xv6 中的 fork() 实现相对直接, 即为子进程重新申请物理页面并复制父进程全部用户内存。这种方式虽然容易理解, 但也让我看到了操作系统实现简单与执行效率之间的权衡。

通过实现 COW, 我认识到 fork() 并不一定需要立即复制所有内存。父子进程首先共享相同的物理页, 只通过页表建立不同的虚拟地址映射, 并利用只读权限限制双方修改。只有真正发生写操作时才创建独立副本, 从而避免大量无意义的内存复制。

本实验与前两个虚拟内存相关实验形成了明显的联系。Page Tables 实验让我理解页表本身如何建立映射; Lazy Allocation 实验利用 page fault 实现“第一次访问时再分配页面”; 而 COW 进一步利用 page fault 实现“第一次写入时再复制页面”。因此我开始认识到 page fault 并不一定意味着程序出现了错误, 它也可以被操作系统主动利用, 成为实现高级内存管理策略的重要机制。

物理页引用计数也是本实验中非常重要的一部分。COW 打破了原有“一个物理页只属于一个地址空间”的简单关系, 使同一个页面能够同时存在于多个进程页表中。因此仅仅通过页表已经无法判断一个物理页面是否可以被释放, 必须额外记录其引用数量。

总体而言, 本实验将 fork()、页表权限、page fault、物理页面共享、引用计数和并发控制等多个知识点结合起来, 使我更加完整地理解了现代操作系统中的 Copy-on-Write 机制。

实验七: Multithreading

1. 实验目的

本实验主要通过实现用户级线程切换、解决多线程哈希表中的并发问题以及实现线程 Barrier 同步机制,加深对线程、上下文切换、锁和条件变量等并发编程机制的理解。

根据实验要求,本实验主要分为三个部分:首先完成用户级线程库中的线程创建和上下文切换;其次使用 POSIX 线程库分析多线程访问共享哈希表时产生的竞态条件,并通过锁保证程序正确性和并行性能;最后使用互斥锁和条件变量实现 Barrier,使多个线程能够在指定位置完成同步。

通过本实验,可以进一步理解线程与进程的区别、CPU 上下文保存和恢复过程、多线程程序中的 race condition,以及锁和条件变量在并发程序中的作用。

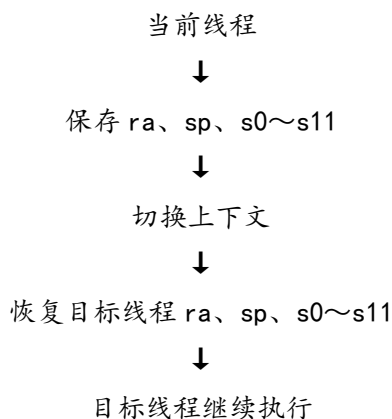
2. 实验内容

2.1 Uthread: 实现用户级线程切换

实验的第一部分要求完成 xv6 提供的用户级线程库。实验代码主要位于 user/uthread.c 和 user/uthread_switch.S 中,其中线程调度框架已经基本给出,需要补充线程创建和线程上下文切换功能。

用户级线程与 xv6 内核中的进程调度具有一定相似性。当一个线程主动让出 CPU 时,需要保存当前线程的执行现场,再恢复下一个可运行线程之前保存的执行现场,使 CPU 从该线程上一次停止的位置继续执行。为此,要在 struct thread 中增加用于保存上下文的结构,主要保存线程切换过程中必须保留的寄存器。根据 RISC-V 函数调用约定,线程切换函数主要保存和恢复 ra、sp 以及 s0~s11 等 callee-saved 寄存器。创建一个新线程时,还需要为其设置独立的栈,并初始化上下文。

在线程调度过程中,thread_schedule()选择下一个状态为 RUNNABLE 的线程,并调用汇编函数 thread_switch(),执行如下逻辑:



2.2 Using threads: 解决多线程哈希表的并发问题

实验第二部分使用 Linux 环境中的 POSIX 线程库,而不是在 xv6 中运行。实验提供的 notxv6/ph.c 实现了一个简单哈希表。在单线程情况下程序能够正确完成数据插入和查询,但当多个线程同时执行 put()时,会出现部分 key 丢失的问题。

首先分别运行 ./ph 1 和 ./ph 2，观察单线程和双线程情况下的执行结果：单线程执行时，哈希表中的 key 能够全部正常保存；双线程并发执行时，则可能出现大量 keys missing。其根本原因是多个线程同时修改同一个哈希 bucket 中的链表时存在 race condition。

因此需要使用 pthread_mutex_t 对共享数据进行保护。最直接的方法是对整个哈希表使用一把全局锁，使每次 put() 操作都在互斥区域内执行。这样可以解决 key 丢失问题，但多个线程的大部分插入操作都会被串行化，并不能充分利用多核 CPU。

为了同时保证正确性和并行性能，进一步采用每个哈希 bucket 一把锁的方式。不同线程操作不同 bucket 时可以同时执行，只有访问同一个 bucket 时才需要互斥。这样既能够保证链表修改的正确性，又可以允许不存在数据冲突的 put() 操作并行执行。

2.3 Barrier：实现多线程屏障同步

实验第三部分要求完成 notxv6/barrier.c 中的 barrier() 函数，实现多个线程之间的屏障同步。

Barrier 的作用是让所有参与同步的线程都到达某一个位置后才能继续执行。实验中主要使用：pthread_mutex_lock()、pthread_mutex_unlock()、pthread_cond_wait()、pthread_cond_broadcast() 实现这一机制。

每个线程进入 barrier() 后，首先获取互斥锁并增加当前到达屏障的线程数量。如果当前线程不是最后一个到达的线程，则使用 pthread_cond_wait() 进入等待状态。该函数在休眠时会暂时释放 mutex，并在被唤醒后重新获得 mutex。当最后一个线程到达 Barrier 时，说明本轮同步条件已经满足，此时需要：（1）清零当前轮次的线程计数；（2）增加 round，表示进入下一轮 Barrier；（3）使用 pthread_cond_broadcast() 唤醒所有等待线程。

由于测试程序会多次循环调用 Barrier，因此不能只实现一次同步，还需要正确区分不同的 Barrier 轮次。官方实验特别要求处理一个速度较快的线程在其他线程尚未完全退出当前 Barrier 时，就重新进入下一轮 Barrier 的情况，因此需要利用 round 变量区分不同同步轮次。

3. 实验过程中遇到的问题及解决

问题一：用户级线程创建成功后无法从指定函数开始执行

在实现 uthread 时，如果只为新线程分配栈并将其状态设置为 RUNNABLE，当调度器第一次切换到该线程时，线程并不能正常开始执行创建时传入的函数。

原因在于上下文切换本质上通过恢复寄存器并执行返回指令继续运行。对于已经运行过的线程，可以通过之前保存的 ra 恢复执行位置；但是对于刚创建的线程，还没有历史执行现场，因此必须人为构造初始上下文。

解决方法是在 thread_create() 中将新线程的 ra 设置为线程入口函数地址，同时将 sp 设置为该线程独立栈空间的栈顶。这样在第一次执行 thread_switch() 恢复新线程上下文时，最终能够从指定线程函数开始执行。

问题二：双线程运行 ph 时出现大量 keys missing

在哈希表实验中，使用单线程运行 ./ph 1 时哈希表能够正确保存全部 key，但按照实验要求增加到两个线程后，会出现大量 keys missing。

分析 put() 和 insert() 的执行过程后发现，多个线程可能同时访问相同的哈希 bucket。假设当前 bucket 的链表头为 A，两个线程同时读取 A：

线程 1 读取 head=A

线程 2 读取 head=A

线程 1 创建节点 B, B->next=A

线程 2 创建节点 C, C->next=A

线程 1 将 head 修改为 B

线程 2 将 head 修改为 C

最终链表变为：

C→A

线程 1 插入的 B 已经丢失。

因此，这并不是哈希算法本身存在问题，而是多个线程对共享数据结构执行“读取—修改—写入”操作时发生了竞态条件。

我的解决方法是在修改同一个 bucket 链表期间获取互斥锁，从而保证同一时刻只有一个线程能够修改该 bucket。修改后再次执行双线程测试，缺失 key 数量能够恢复为 0。

问题三：Barrier 在连续多轮使用时可能发生不同轮次之间的线程混淆

实现最基本的 Barrier 后，如果只使用 nthread 记录当前到达数量，在程序重复调用 Barrier 时仍然可能出现竞态问题。例如某个线程执行速度较快，在当前轮被唤醒后立即完成后续代码并再次调用 barrier()，而上一轮中的其他线程可能刚刚被唤醒、尚未完全离开 Barrier。此时如果直接复用同一个 nthread 计数，就可能将下一轮到达的线程错误计入上一轮，导致同步逻辑混乱。

因此实验中需要使用 round 表示当前 Barrier 轮次。线程进入 Barrier 时记录当前轮次。如果不是最后一个线程，则需等待当前 round 发生变化。最后一个线程到达后再执行：nthread=0; round++; broadcast()。这样即使某个线程很快重新进入 Barrier，也能够通过 round 区分自己属于下一轮同步，不会与上一轮线程发生混淆。

4. 实验心得

Multithreading 实验使我从之前主要关注操作系统中的进程和虚拟内存，进一步进入了并发执行和线程同步领域。实验中的三个部分分别从线程切换、共享数据保护和线程同步三个角度展示了多线程程序中最核心的问题。

通过 uthread 部分，我认识到线程切换的本质与进程上下文切换十分相似。一个线程能够“暂停后继续执行”，并不是因为程序代码本身记住了执行位置，而是因为系统保存了 ra、sp 以及 callee-saved 寄存器等 CPU 状态。当这些状态重新恢复后，CPU 就能够继续执行该线程之前的代码。

哈希表实验让我直观地观察到了 race condition。单线程程序完全正确，并不意味着同一段代码放到多线程环境中仍然正确。两个线程之间只要存在某种特定的指令交叉顺序，就可能使共享数据结构遭到破坏。

同时，从全局锁优化到 bucket lock 的过程使我认识到并发程序不仅要考虑正确性，还必须考虑并行性。锁的范围过大虽然容易保证正确，但也可能让多个线程重新退化成串行执

行。合理降低锁粒度，可以让不存在冲突的操作真正并行进行。

Barrier 实验则让我进一步理解了线程之间的同步关系。锁主要用于保证临界区互斥，而条件变量允许线程等待某个状态发生变化。通过将 **mutex**、**condition variable**、计数器和 **round** 结合起来，可以实现多个线程之间较复杂的执行顺序控制。

总体而言，本实验让我认识到，多线程程序的难点并不只是创建多个线程，而是如何保证多个执行流在共享资源和复杂执行顺序下仍然保持正确。通过完成用户线程切换、哈希表锁优化和 **Barrier** 同步，我对上下文切换、**race condition**、锁粒度、条件变量以及线程同步形成了更加系统的认识。

实验八：Locks

1. 实验目的

本实验主要通过分析并优化 xv6 内核中的锁竞争问题，加深对多核并发、spinlock、共享数据结构以及锁粒度设计的理解。

xv6 运行在多核环境中时，不同 CPU 可能同时访问物理内存分配器、buffer cache 等共享资源。为了保证数据结构的一致性，内核需要使用锁进行同步，但如果多个 CPU 频繁竞争同一把锁，就会造成严重的性能下降。

因此，本实验的主要目标是在保证内核正确性的前提下，减少不同 CPU 之间对共享锁的竞争。实验按照教程主要分为两个部分：首先修改物理内存分配器，为不同 CPU 建立独立的空闲页链表；随后重新设计 buffer cache 的锁机制，通过哈希桶降低对全局锁的竞争。

通过本实验，可以进一步理解锁竞争产生的原因，以及如何通过数据结构划分、降低锁粒度和局部化共享资源来提升多核程序的并行性能。

2. 实验内容

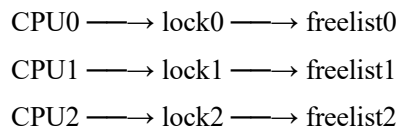
2.1 优化物理内存分配器

原始 xv6 使用一个全局的 kmem 结构管理所有空闲物理页面，其中只有一条全局 freelist，同时通过一把 kmem.lock 保护整个空闲链表。其基本结构可以表示为：



当多个 CPU 同时调用 kalloc()或 kfree()时，都需要竞争同一把锁。在多核环境下，这会造成大量锁等待，使物理内存分配器成为明显的并发瓶颈。

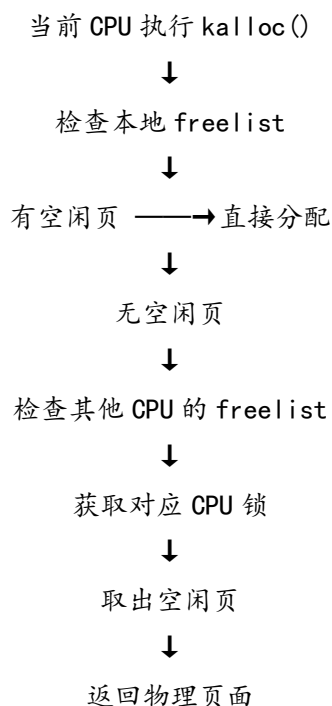
因此，本实验首先将原来的单一空闲链表改为每个 CPU 分别维护一个空闲页链表。每个 CPU 对应自己的锁和 freelist，正常情况下只操作当前 CPU 自己的内存链表。修改后的结构可以表示为：



在 kfree()中，通过 cpuid()获得当前 CPU 编号，将释放的物理页加入当前 CPU 对应的空闲链表。

在 kalloc()中，首先尝试从当前 CPU 自己的空闲链表中申请物理页。由于不同 CPU 通常只访问各自的链表，因此多个 CPU 可以同时执行内存分配，而不再频繁竞争同一把全局锁。

但是，不同 CPU 上的内存使用量可能并不均衡。当某个 CPU 自己的空闲链表已经为空，而其他 CPU 仍然保存大量空闲页时，如果直接返回分配失败，会造成物理内存资源浪费。因此还需要实现内存“窃取”机制。当当前 CPU 没有空闲页时，依次检查其他 CPU 的空闲链表，并从其中取出部分或一个空闲页供当前 CPU 使用。其基本过程为：



通过这种设计，大多数内存分配和释放操作只使用当前 CPU 自己的锁，只有本地内存不足时才需要访问其他 CPU 的数据，从而显著降低 kmem 锁的竞争。

2.2 优化 buffer cache

实验第二部分主要优化 xv6 的 buffer cache。

buffer cache 用于缓存磁盘块。当文件系统需要读取某个磁盘块时，首先在 buffer cache 中寻找是否已经存在相应的数据。如果命中缓存，则可以直接使用内存中的 buffer，而不需要再次访问磁盘。

原始 xv6 中的 buffer cache 使用一条双向链表保存所有 struct buf，并通过一把全局的 bcache.lock 保护整个缓存结构。这种设计实现简单，但在多核环境下，当多个进程同时执行文件系统操作时，都会频繁调用 bget()和 brelse()。即使两个 CPU 访问的是完全不同的磁盘块，也必须竞争同一把 bcache.lock，因此会产生严重的锁竞争。

为减少这种竞争，本实验重新设计 buffer cache 的数据结构，将 buffer 按照 blockno % NBUCKET 进行哈希，并分配到多个 bucket 中。实验实现中采用多个哈希桶，并为每个 bucket 分别设置一把锁。

当 bget()查找某个磁盘块时，首先根据 blockno 计算其对应 bucket，只需要获取该 bucket 的锁，然后在桶内查找是否已经存在目标 buffer。如果缓存命中，则增加 buffer 的引用计数，并返回该 buffer；如果缓存未命中，则需要寻找一个当前没有被使用的 buffer，并将其重新分配给新的磁盘块。这一过程涉及 buffer 从原 bucket 移动到另一个 bucket，因此相比单纯查找更加复杂。

为了保证 buffer 替换过程中的正确性，在 cache miss 或 buffer replacement 时使用额外的锁对替换过程进行保护，而普通缓存命中只需要获取对应的 bucket lock。这种设计使最常见的 buffer 查找操作可以在不同 bucket 之间并行进行，从而降低全局锁的竞争。

此外，brelse()也需要根据当前 buffer 所在 bucket 获取对应的 bucket lock，并安全地减少 refcnt。只有当 buffer 引用计数降为 0 以后，它才可以在后续 cache miss 中被重新使用。

3. 实验过程中遇到的问题及解决

问题一：修改 buffer cache 为哈希桶结构后，bcachetest 无法正常通过

在完成 buffer cache 哈希化的初步实现后，实际运行 bcachetest 时出现测试异常，说明新的缓存管理逻辑虽然能够完成基本编译，但 buffer 查找、替换或锁管理仍然存在问题。

经过检查发现，原始 buffer cache 只有一把全局锁，因此 buffer 查找、引用计数修改和 buffer 替换全部在同一个临界区中完成。改成多个 bucket lock 以后，同一个操作可能涉及多个锁，原本由全局锁保证的原子性已经不存在。特别是在 cache miss 情况下，需要寻找一个 refcnt==0 的 buffer，并可能将其从原来的 bucket 移动到新的 bucket。如果这一过程只持有目标 bucket 的锁，其他 CPU 可能同时选中同一个空闲 buffer，或者在移动 buffer 过程中访问到中间状态，从而破坏 buffer cache 的一致性。

因此我重新整理了锁的职责：

- 普通 buffer 查找只使用对应的 bucket_lock；
- cache miss 和 buffer replacement 使用额外的全局替换锁进行串行化；
- buffer 移动过程中正确获取相关 bucket lock；
- buffer 的 dev、blockno、valid 和 refcnt 等信息在受保护状态下完成修改。

4. 实验心得

Locks 实验让我第一次从性能角度系统地分析 xv6 内核中的并发问题。之前的 Multithreading 实验更多关注多线程程序如何保证正确性，而本实验进一步提出了一个新的问题：即使加锁以后程序已经正确，如果所有 CPU 仍然频繁竞争同一把锁，那么多核并行带来的性能优势也会被大幅削弱。

物理内存分配器部分让我理解了“每 CPU 数据结构”这一常见的内核优化思想。原始的全局空闲链表虽然简单，但所有 CPU 都会竞争同一个 kmem.lock。将空闲页拆分到每个 CPU 自己的链表后，大部分内存分配和释放操作都可以在本地完成，从而显著降低共享访问频率。同时，内存窃取机制也让我认识到局部化资源管理并不能完全取代全局资源共享。不同 CPU 之间的内存需求不可能始终均衡，因此仍然需要在本地资源耗尽时允许跨 CPU 获取页面。

buffer cache 部分的实现难度明显更高。原来的全局锁虽然竞争严重，但同时也隐式保证了大量操作的原子性。当将它拆分成多个 bucket lock 以后，就必须重新考虑 buffer 查找、引用计数、缓存替换、buffer 迁移以及同一磁盘块唯一缓存等问题。

通过从全局锁优化为 bucket lock，我进一步理解了锁粒度的概念。锁粒度越大，实现通常越简单，但并行性能越差；锁粒度越小，可以允许更多操作同时进行，但同步逻辑也会更加复杂。操作系统设计需要在正确性、复杂性和性能之间进行平衡。

总体而言，本实验让我对 spinlock、多核并发、锁竞争、锁粒度、死锁以及共享数据结构的一致性有了更加深入的理解。也让我认识到，在操作系统内核中，性能优化往往不是单纯减少代码执行次数，而是尽可能减少 CPU 之间对共享状态的争用。

实验九：File System

1. 实验目的

本实验主要通过扩展 xv6 文件系统的大文件支持能力并实现符号链接机制，加深对 inode、磁盘块映射、目录路径解析以及文件系统系统调用的理解。

按照实验教程要求，本实验主要包含两个任务：第一部分通过增加二级间接块，使 xv6 能够支持更大的文件；第二部分实现 symlink 系统调用以及符号链接解析功能。

通过本实验，可以进一步理解文件逻辑块号到磁盘物理块号的映射过程，掌握直接块、一级间接块和二级间接块之间的组织关系，同时理解符号链接与普通文件、硬链接之间的区别，以及 open() 进行路径解析时的具体处理流程。

2. 实验内容

2.1 Large files: 实现大文件支持

原始 xv6 文件系统中，一个 inode 包含 12 个直接块地址和 1 个一级间接块地址。每个磁盘块大小为 1024 字节，而一个一级间接块可以保存 256 个磁盘块地址，因此原始 xv6 文件最多只能包含 268 个数据块。官方提供的 bigfile 测试需要创建包含 65803 个数据块的大文件，因此需要增加二级间接块结构。

为了在不改变磁盘 inode 大小的情况下增加二级间接块，需要将原来的 12 个直接块减少为 11 个，然后将 inode 中的地址数组重新组织为：

addr[0]~addr[10]



11 个直接块

addr[11]



一级间接块



最多 256 个数据块

addr[12]



二级间接块



256 个一级间接块



每个一级间接块最多指向 256 个数据块

因此修改后的文件最大数据块数量为 $11+256+256\times 256=65803$ ，符合测试要求。

实验首先修改 fs.h 中的 NDIRECT、NINDIRECT 和 MAXFILE 等定义，并保证磁盘 inode

中的 `struct dinode` 与内存 `inode` 中的 `struct inode` 所包含的 `addrs[]` 数组长度保持一致。修改 `inode` 格式以后需要重新生成 `fs.img`，否则旧文件系统镜像与新的 `inode` 结构可能不匹配。

随后重点修改 `fs.c` 中的 `bmap()`。`bmap()` 的作用是将文件内部的逻辑块号转换为实际磁盘块号，同时在写文件过程中根据需要动态分配新的数据块。

除了修改 `bmap()` 之外，还必须同步修改 `itrunc()`。当文件被删除或者长度被截断时，不仅需要释放直接块和一级间接块，还需要遍历二级间接块中的每一个一级间接块，将所有数据块、一级间接块和二级间接块本身依次释放。

2.2 Symbolic links: 实现符号链接

实验第二部分要求在 `xv6` 中加入符号链接机制。

符号链接本身是一种特殊类型的文件，其内容不是普通数据，而是另一个文件的路径。当用户通过符号链接执行 `open()` 时，内核需要读取符号链接保存的目标路径，并继续查找真正的目标文件。

首先增加新的系统调用 `symlink(char *target, char *path)`。其中 `target` 表示符号链接所指向的目标路径，`path` 表示新创建的符号链接本身的路径。按照普通系统调用的添加流程，需要增加系统调用编号、用户态声明以及系统调用入口，并在 `sysfile.c` 中实现 `sys_symlink()`。

随后在文件类型定义中增加 `T_SYMLINK`，使文件系统能够通过 `inode` 的 `type` 字段识别符号链接。同时增加 `O_NOFOLLOW` 打开标志。当用户使用该标志调用 `open()` 时，内核应该直接打开符号链接文件本身，而不是继续解析其指向的目标。

创建符号链接时，可以使用普通 `inode` 的数据块保存目标路径。`sys_symlink()` 首先通过 `create()` 创建一个类型为 `T_SYMLINK` 的新 `inode`，再使用 `writei()` 将 `target` 字符串写入该 `inode`。

本部分最主要的修改位于 `sys_open()`。原始 `open()` 在通过 `namei()` 获得 `inode` 后，会直接根据文件类型和打开方式创建文件描述符。增加符号链接以后，如果 `namei()` 得到的是 `T_SYMLINK` 类型 `inode`，并且用户没有指定 `O_NOFOLLOW`，就需要读取该符号链接保存的路径，然后再次执行 `namei()` 查找目标文件。如果目标文件本身仍然是符号链接，则需要继续解析，直到找到一个普通文件。

3. 实验过程中遇到的问题及解决

问题一：修改 `inode` 直接块数量后，文件系统镜像与新的 `inode` 结构不一致

实现大文件功能时，需要将直接块数量由 12 个调整为 11 个，并重新定义 `MAXFILE`。修改这些参数以后，如果继续直接使用此前生成的 `fs.img`，可能出现文件系统运行异常。这是因为 `mkfs` 在生成文件系统镜像时同样会使用 `NDIRECT` 以及 `inode` 相关定义，旧镜像中的数据布局与新代码中的 `inode` 解释方式可能不一致。

因此在修改 `inode` 结构后必须执行 `make clean`，删除旧的编译结果和文件系统镜像，再重新编译生成新的 `fs.img`。

问题二：只修改 `bmap` 而没有完整修改 `itrunc` 会导致磁盘块无法正确释放

完成 `bmap()` 以后，文件可以通过二级间接块继续增长，但如果没有同步修改 `itrunc()`，删除大文件时就只能释放原有的直接块和一级间接块。

正确的释放方法是必须按照与分配相反的方向进行遍历。对于每一个二级间接项，需要先读取其指向的一级间接块，再逐个释放其中的数据块；随后释放一级间接块本身；所有一

级间接块都处理完成后，最后释放二级间接块。如果遗漏其中任意一层，就会导致磁盘块虽然不再被文件使用，却没有重新加入空闲块集合，形成磁盘空间泄漏。

同时，所有通过 `bread()` 获取的 `buffer` 在使用完成后都必须调用 `brelse()` 释放，否则还会造成 `buffer cache` 资源无法正常回收。

4. 实验心得

File System 实验让我对 xv6 文件系统的理解从之前主要使用 `open()`、`read()` 和 `write()` 等接口，进一步深入到文件数据在磁盘上的实际组织方式。

在 Large files 部分，通过修改 `bmap()`，我更加清楚地理解了 `inode` 并不是直接保存整个文件的数据，而是保存磁盘块的索引信息。修改 `itrunc()` 则使我认识到文件系统中的资源回收与资源分配同样重要。创建一个二级索引结构不仅要知道如何逐层分配，还必须能够在文件删除时按照正确层级完整释放所有数据块和索引块。

Symbolic links 部分让我进一步理解了路径解析机制。符号链接表面上只是一个新的文件类型，但真正实现以后会直接影响 `open()` 的路径查找过程。内核需要在获取 `inode` 之后判断它是否为符号链接，再读取其中保存的目标路径，并重新进行路径解析。

总体而言，本实验将 `inode`、磁盘块分配、`buffer cache`、日志、路径解析和系统调用等多个文件系统知识联系起来，使我对一个文件从用户执行 `open()` 开始，到内核查找 `inode`，再到 `bmap()` 定位磁盘块的完整过程有了更加系统的认识。

实验十：mmap

1. 实验目的

本实验主要通过 xv6 中实现 `mmap()` 和 `munmap()` 系统调用，加深对虚拟内存、文件系统、page fault 以及进程地址空间管理之间关系的理解。

`mmap()` 可以将文件内容映射到进程的虚拟地址空间，使用户程序能够像访问普通内存一样访问文件数据。与普通 `read()` 相比，`mmap()` 并不会在系统调用执行时立即把整个文件读入内存，而是先建立一段虚拟地址区域，在用户真正访问某一页时通过 page fault 按需从文件中读取数据。

按照实验教程要求，本实验主要完成文件映射区域的建立、基于 page fault 的惰性加载、`munmap()` 解除映射和共享映射的数据写回，并进一步处理进程退出和 `fork()` 时 VMA 的释放与继承。

通过本实验，可以进一步理解 Lazy Allocation、文件系统和虚拟内存之间的联系，并掌握 VMA、页表映射、文件引用计数以及共享文件映射等机制。

2. 实验内容

2.1 实现 mmap 并建立 VMA

实验首先需要增加 `mmap()`、`munmap()` 两个系统调用，并为每个进程维护若干个 VMA 结构，用于记录当前进程中已经建立的文件映射区域。

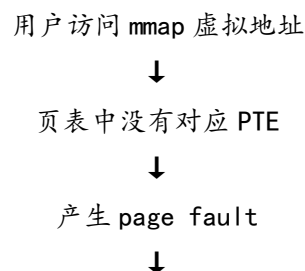
当用户调用 `mmap()` 时，内核首先解析系统调用参数，并检查文件权限是否满足映射要求。例如，如果使用 `MAP_SHARED` 建立可写映射，则对应文件本身必须允许写入。

实验采用较简单的地址选择方式，从用户地址空间中选择一段尚未使用的虚拟地址作为映射区域，然后更新相应 VMA 信息。`mmap()` 在执行时只是记录虚拟地址范围、对应的文件、映射权限和类型，不立即分配物理页。这样即可实现 Lazy mmap，即真正访问页面时再完成物理页分配和文件数据加载。

同时，由于 VMA 中保存了文件指针，而原有文件描述符之后可能被用户关闭，因此在建立 VMA 时需要调用 `fildup()` 增加文件引用计数，使映射区域在原文件描述符关闭之后仍然能够继续正常使用。

2.2 通过 page fault 按需加载文件页面

`mmap()` 只创建 VMA 而不立即分配物理页，因此当用户第一次访问映射区域时，对应页表项尚不存在，CPU 会产生 page fault。因此需要修改 `usertrap()`，在发生 load page fault 或 store page fault 时判断缺页地址是否位于某个有效 VMA 中。整体处理过程为：





在加载页面时，需要首先将发生缺页的地址通过 `PGROUNDDOWN()` 转换为页边界，然后计算该页面相对于 VMA 起始地址的偏移量。随后调用文件系统相关函数将对应文件内容读入新分配的物理页面。

建立页表映射时，需要根据用户传给 `mmap()` 的权限设置 PTE。并增加 `PTE_U` 使用户程序能够访问该页面。通过这种方式，文件页面只有在用户真正访问时才会加载到物理内存中，从而实现按需分页。

2.3 实现 `munmap` 并处理共享映射写回

实验下一部分需要实现 `munmap()`，使用户能够解除部分或全部文件映射。

执行 `munmap()` 时，首先找到与给定虚拟地址对应的 VMA，然后遍历需要解除映射的页面。对于已经实际加载的页面，需要解除页表映射并释放对应物理内存；对于从未被访问、因此根本不存在 PTE 的页面，则应直接跳过，而不能因为页面未映射就产生 `panic`。

映射解除时是否写回视实际情况而定。如果映射类型为 `MAP_SHARED` 并且该页面允许写入，则用户对映射页面的修改需要写回原文件。而对于 `MAP_PRIVATE` 映射，用户对内存的修改只属于当前进程，不需要写回原始文件。

2.4 处理 `exit` 和 `fork` 中的 VMA

完成基本 `mmap()` 和 `munmap()` 以后，还需要处理进程生命周期对映射区域的影响。

首先，在进程调用 `exit()` 时，所有仍然存在的 VMA 都必须被解除。这意味着内核需要遍历当前进程的 VMA 数组，将已经加载的页面按照 `munmap()` 类似的方式释放；如果属于需要写回的 `MAP_SHARED` 映射，还应当将修改内容写回文件。最后关闭 VMA 对应的文件引用。

其次，需要修改 `fork()`。子进程应该继承父进程的 VMA 信息，因此需要复制父进程的 VMA 数组，并对每一个正在使用的 VMA 调用 `filedup()` 增加文件引用计数。

该实验并不要求父子进程立即共享完全相同的物理 `mmap` 页面，因此子进程可以继续采用 Lazy Allocation 方式。在 `fork()` 之后，只需要保证 VMA 描述信息正确继承；当子进程真正访问映射地址时，再由其 `page fault` 处理程序独立加载相应文件页面。

3. 实验过程中遇到的问题及解决

问题一：初步实现 `mmap` 后，`mmaptest` 多个测试项均无法正常执行

在完成 `mmap()` 和 `munmap()` 系统调用的初步框架后，运行 `mmaptest` 时曾出现多个子测试无法正常继续，部分情况下表现为程序执行异常甚至无法进入后续测试。

经过检查发现，`mmap` 实验并不是只需要增加两个系统调用接口。一个完整的文件映射还依赖：VMA 初始化、page fault 处理、文件引用计数、页表权限、`munmap` 资源释放、`exit` 清理、`fork` 继承等多个部分。

如果只完成系统调用入口和 VMA 记录，而没有完整实现 page fault 加载，用户第一次访问映射区域时就会因为页表中没有对应 PTE 而被内核终止。

因此，我重新按照官方实验流程逐项完善实现，使 `mmap()` 只负责登记 VMA，而 page fault 负责真正加载物理页面，再逐步补充 `munmap()`、`exit()` 和 `fork()` 逻辑。

问题二：mmap read/write 和 mmap dirty 测试失败，并出现 log_write outside of trans

这一错误说明，在将 `MAP_SHARED` 页面内容写回文件时，直接调用了文件系统写操作，但当前代码没有处于有效的文件系统日志事务中。

xv6 文件系统为了保证磁盘操作的一致性，修改文件内容时必须使用日志系统。`writei()` 等底层操作最终可能调用 `log_write()`，而 `log_write()` 要求当前执行路径已经通过 `begin_op()` 开启事务，并在完成后通过 `end_op()` 结束事务。

因此，在 `munmap()` 或进程退出过程中对共享映射页面执行文件写回时，需要按照文件系统规定正确建立日志事务，而不能直接在任意位置调用底层写函数。需要将共享映射写回操作放入合法的事务范围中，这样就可以有效避免再次触发 `log_write outside of trans`。

4. 实验心得

`mmap` 实验是整个 xv6 虚拟内存相关实验中综合性非常强的一项实验。它同时涉及页表、page fault、文件系统、日志、文件引用计数、进程创建和退出等多个模块，因此实现过程比 Lazy Allocation 和 Copy-on-Write 更加复杂。

通过本实验，我首先更加深入地理解了 VMA 的作用。页表只能描述当前已经建立的虚拟地址到物理地址映射，而 VMA 描述的是一段更高层次的虚拟地址区域及其语义。即使某个 `mmap` 页面当前还没有 PTE，只要它属于合法 VMA，后续 page fault 仍然可以根据 VMA 信息动态建立映射。

实验过程中出现的 `log_write outside of trans` 也让我进一步认识到不同内核模块之间的规则不能被忽略。即使 `mmap` 主要属于虚拟内存功能，只要最终需要修改文件，就必须遵守 xv6 文件系统的日志事务机制。

最后，通过修改 `exit()` 和 `fork()`，我进一步体会到操作系统中资源生命周期管理的重要性。任何增加到进程中的新资源，都需要思考它在进程创建、复制和退出时应该如何处理，否则局部功能即使能够正常运行，也会在复杂测试中出现资源泄漏或状态不一致。

总体而言，本实验将之前 Page Tables、Traps、Lazy Allocation、Copy-on-Write 和 File System 实验中的知识进行了综合应用，使我对 xv6 中虚拟内存和文件系统之间的联系形成了更加完整的认识，也进一步提高了分析跨模块内核问题的能力。

实验十一：Networking

1. 实验目的

本实验主要通过完成 xv6 中的 E1000 网卡驱动，使操作系统能够通过 QEMU 模拟的网络设备发送和接收以太网数据包，加深对设备驱动、中断机制、DMA、描述符环形队列以及操作系统网络 I/O 过程的理解。

xv6-2020 的 Networking 实验使用 QEMU 模拟 Intel82540EM 系列 E1000 网卡。实验代码已经提供 E1000 初始化过程以及简单的 ARP、IP 和 UDP 协议栈，主要任务是在 kernel/e1000.c 中完成 e1000_transmit() 和 e1000_recv(), 使网卡能够正确发送和接收网络数据包。

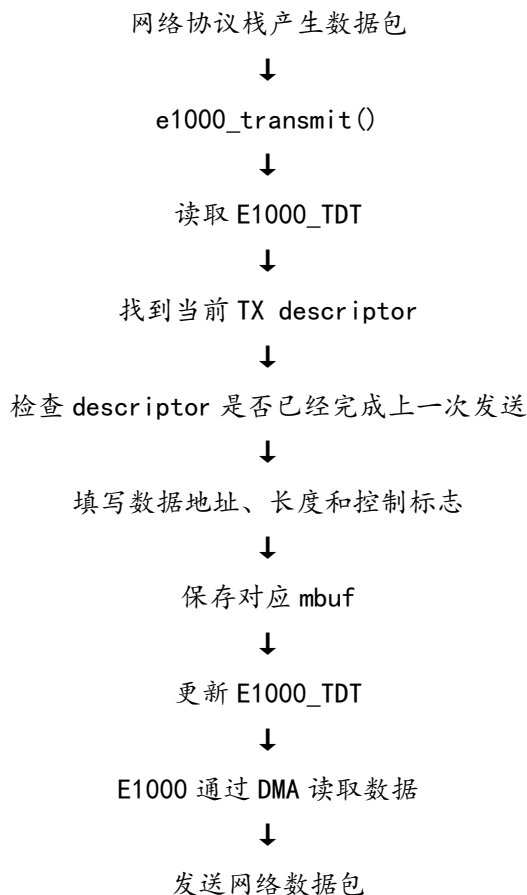
通过本实验，可以进一步理解 CPU、内存和外部设备之间的交互方式，掌握 DMA 和中断驱动 I/O 的基本工作过程，并将前面 Traps 和 Locks 实验中的中断处理、锁和多核并发知识应用到实际设备驱动程序中。

2. 实验内容

2.1 实现 E1000 数据包发送

E1000 使用 Transmit Ring 管理待发送的数据包。Transmit Ring 由多个 TX descriptor 组成，每个 descriptor 记录一个待发送数据包在内存中的地址、数据长度以及控制和状态信息。

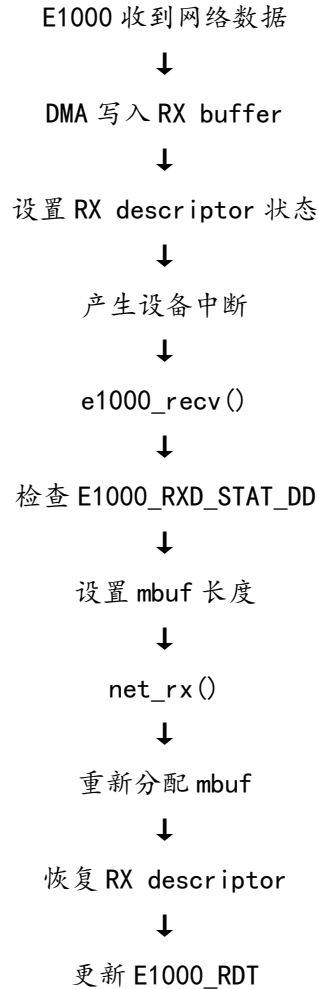
完整发送过程可以概括如下：



由于多个进程可能同时通过网络协议栈发送数据，因此还需要使用锁保护发送队列，防止不同 CPU 同时修改同一个 TX descriptor 或者 E1000_TDT。

2.2 实现 E1000 数据包接收

E1000 的数据接收同样通过环形描述符队列实现。整个流程可以概括为：



由于 RX Ring 本身是循环结构，当处理位置到达数组末尾后，需要通过模运算重新回到第一个 descriptor，保证连续接收的数据包数量可以超过 RX Ring 本身的长度。实验教程特别要求实现能够正确处理累计接收数据包超过 16 个 descriptor 的情况。

2.3 完成网络中断处理并进行测试

E1000 接收到数据包后会产生外部设备中断，因此除了完成 RX Ring 处理外，还需要保证 xv6 能够识别 E1000 对应的 PLIC 中断，并进入网卡中断处理函数。

在 devintr() 中，根据 PLIC 返回的 IRQ 判断中断来源。当检测到 E1000_IRQ 时调用 e1000_intr()，由 E1000 中断处理程序进一步完成网卡中断确认以及数据包接收。

3. 实验过程中遇到的问题及解决

问题一：没有检查 TX descriptor 的 DD 状态会造成未发送数据被覆盖

实现 e1000_transmit() 时，如果每次读取 E1000_TDT 以后立即修改对应 descriptor，而没有检查 E1000_TXD_STAT_DD 状态位，就可能在 E1000 仍然处理上一个数据包时覆盖 descriptor。因此在使用 descriptor 之前必须首先检查 DD 标志。只有该标志已经由硬件设置

时，才能说明上一次发送操作真正完成。如果 descriptor 尚未完成发送，则 e1000_transmit() 直接返回-1，由调用者处理对应的 mbuf。

问题二：发送数据后立即释放 mbuf 会造成 DMA 读取已经失效的内存

在普通函数调用中，当某个函数完成数据处理以后通常可以立即释放相关内存，但 E1000 的数据发送过程并不是同步完成的。驱动将数据地址写入 TX descriptor 以后，真正读取该数据的是 E1000 硬件。CPU 更新 E1000_TDT 之后即可继续执行其他代码，而 E1000 会在之后通过 DMA 访问该内存。因此如果在 e1000_transmit() 返回之前直接调用 mbuf_free(m)，就可能发生 DMA 读取到错误数据。

解决方法是为每个 TX descriptor 保存对应的 mbuf 指针。只有下一次重新使用该 descriptor，并且确认其中 DD 已经设置时，才能释放此前的数据缓冲区。

4. 实验心得

Networking 实验是 xv6 全部实验中与硬件设备交互最直接的一项实验。前面的实验主要围绕进程、虚拟内存、锁和文件系统等操作系统内部机制，而本实验需要让内核真正通过模拟网卡与外部环境进行数据交换，因此使我对设备驱动在操作系统中的作用有了更加直观的理解。

通过分析 E1000 的发送和接收过程，我认识到了 DMA 的重要作用。CPU 并不会逐字节将网络数据复制到网卡，而是将数据缓冲区的物理地址写入 descriptor，然后由 E1000 直接从内存读取或向内存写入数据。CPU 只需要负责准备 descriptor 并处理完成事件，大量数据传输工作由设备自行完成，从而降低 CPU 参与 I/O 的开销。

TX Ring 和 RX Ring 的实现也让我理解了环形描述符队列的设计思想。驱动程序和硬件并不是通过一次函数调用同步完成数据交换，而是利用共享内存中的 descriptor 以及设备寄存器进行协作。驱动不断生产待发送 descriptor 或者回收接收 descriptor，而 E1000 则异步消费和填充这些 descriptor。

完成本实验后，我对操作系统设备驱动、DMA、中断、环形队列和网络协议栈之间的关系形成了更加完整的认识。

总结与收获

xv6-2020 的十一个实验覆盖了 Unix 用户程序、系统调用、页表、Trap、惰性内存分配、Copy-on-Write、多线程、锁、文件系统、mmap 以及网络驱动等内容，基本串联起了一个操作系统从用户程序到底层硬件的主要组成部分。通过完成这十一个实验，我对操作系统的理解从课本中的概念逐渐转变为对真实内核代码运行机制的理解。

除了知识本身，整个实验过程中对我提升最大的一点是解决问题的方法。

xv6 实验中的很多问题并不会直接告诉我“哪一行代码写错了”。例如，我遇到过页表修改后程序异常、buffer cache 测试失败、mmap 测试无法通过、log_write outside of trans、COW 引用计数错误以及网卡中断无法正确进入处理流程等问题。这些问题往往涉及多个函数甚至多个模块，仅仅根据报错信息很难直接定位。

在不断调试的过程中，我逐渐形成了一套比较清晰的问题分析方法：首先根据报错或测试结果判断问题发生在哪个阶段，再结合调用流程缩小范围；随后分析当前实现破坏了原系统中的哪个假设或不变量；最后再通过源码、测试程序和打印信息验证自己的判断。

完成全部实验后，我对操作系统本身也产生了更加具体的认识。最初学习操作系统时，进程、虚拟内存、文件系统、中断、锁等知识往往以不同章节独立出现，很容易将它们理解成相互分离的知识点。但在 xv6 中真正修改代码以后，我发现这些模块之间实际上有着非常紧密的联系。一次普通的用户程序执行可能经历：用户程序、系统调用或异常、Trap 进入内核、进程和页表、内存分配、文件系统或设备驱动、中断完成 I/O、重新调度、返回用户程序这整个过程。这让我逐渐意识到，操作系统并不是由许多互不相关的功能堆积起来的软件，而是一个需要维持大量状态和不变量的整体系统。

同时，操作系统还必须不断处理各种矛盾：既要隔离不同进程，又要允许它们共享资源；既要保证并发正确，又要尽量减少锁竞争；既要保证数据安全，又要提高 I/O 性能；既要充分利用物理内存，又要让每个程序感觉自己拥有独立而连续的地址空间。

因此，在完成这些实验以后，我认为操作系统的核心并不仅仅是实现若干功能，而是通过抽象、隔离、共享和调度，在有限且复杂的硬件资源之上建立一个稳定、高效、可控的执行环境。